

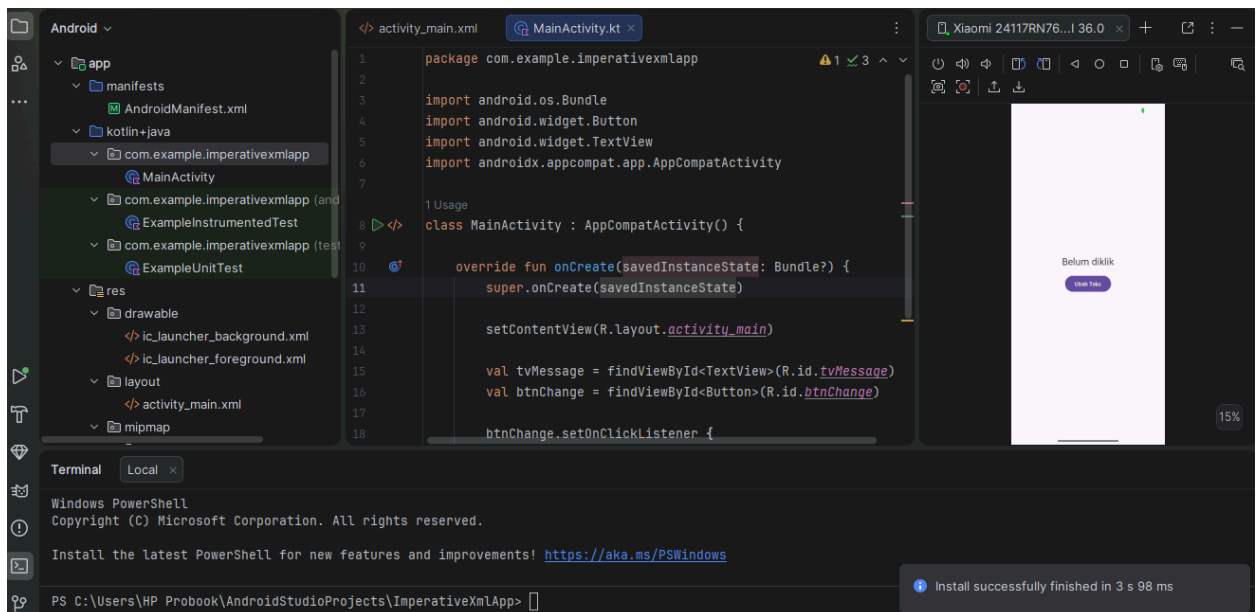
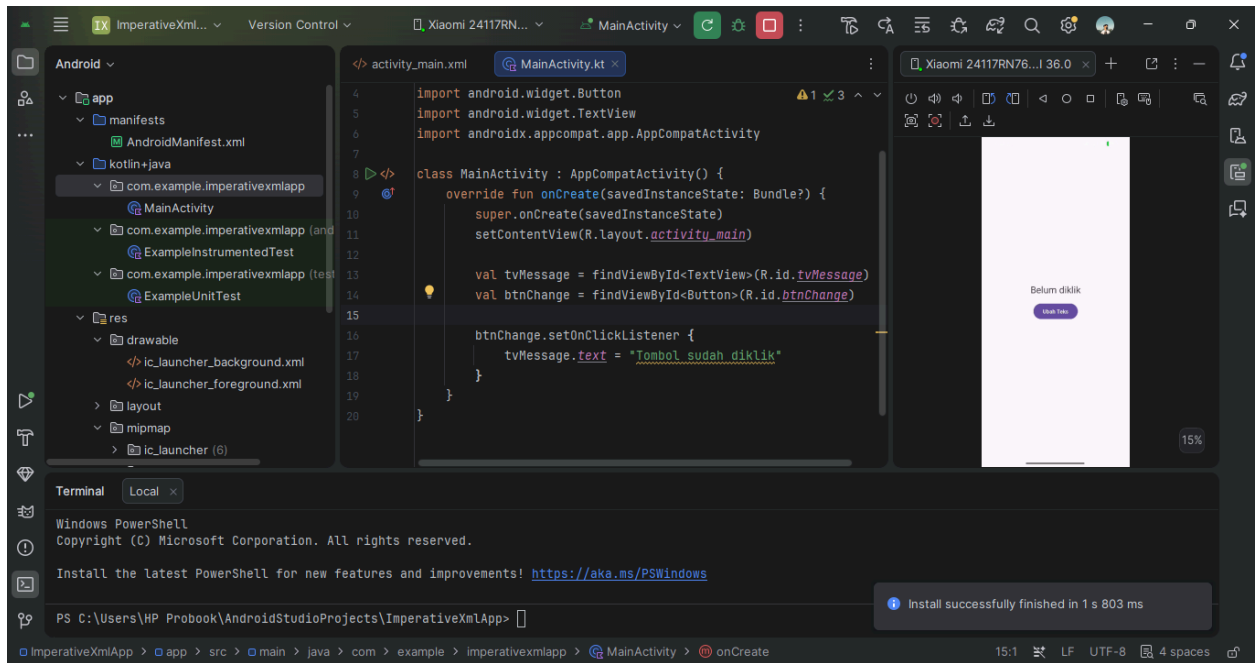
Nama : Steven Elyas Geraldo Siringoringo

Kelas : PAPB – A

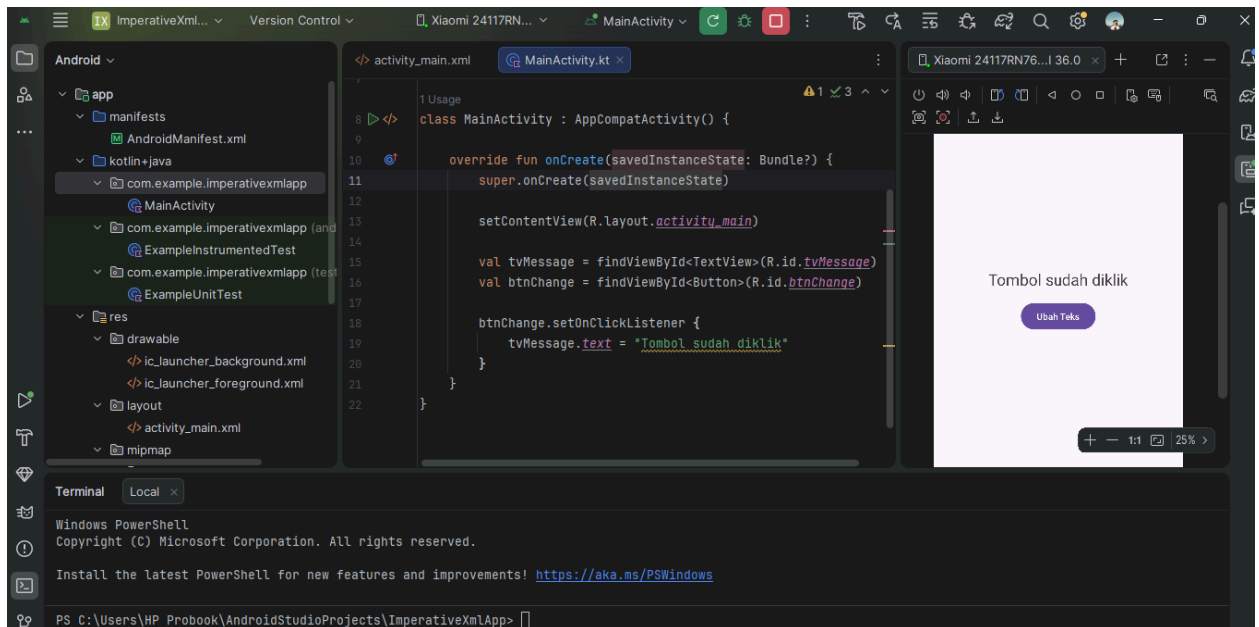
NIM : 245150607111025

Link Github : <https://github.com/stvendevs/TicketApp.git>

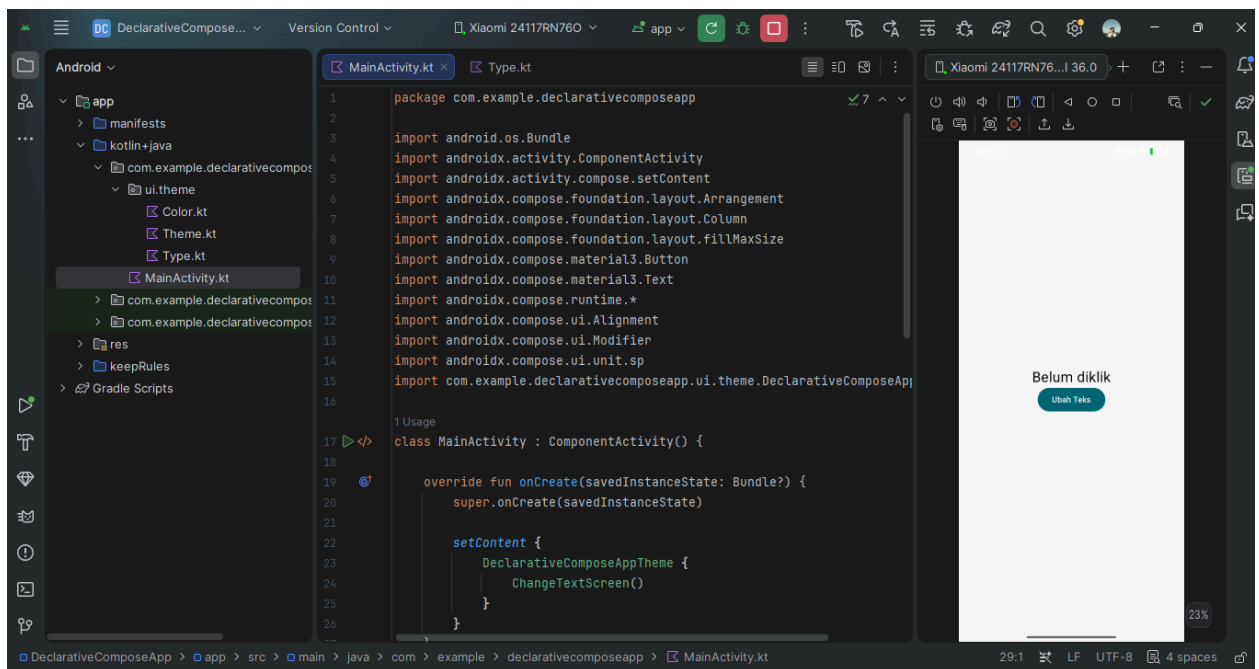
1. Project A - XML/View (Imperatif)

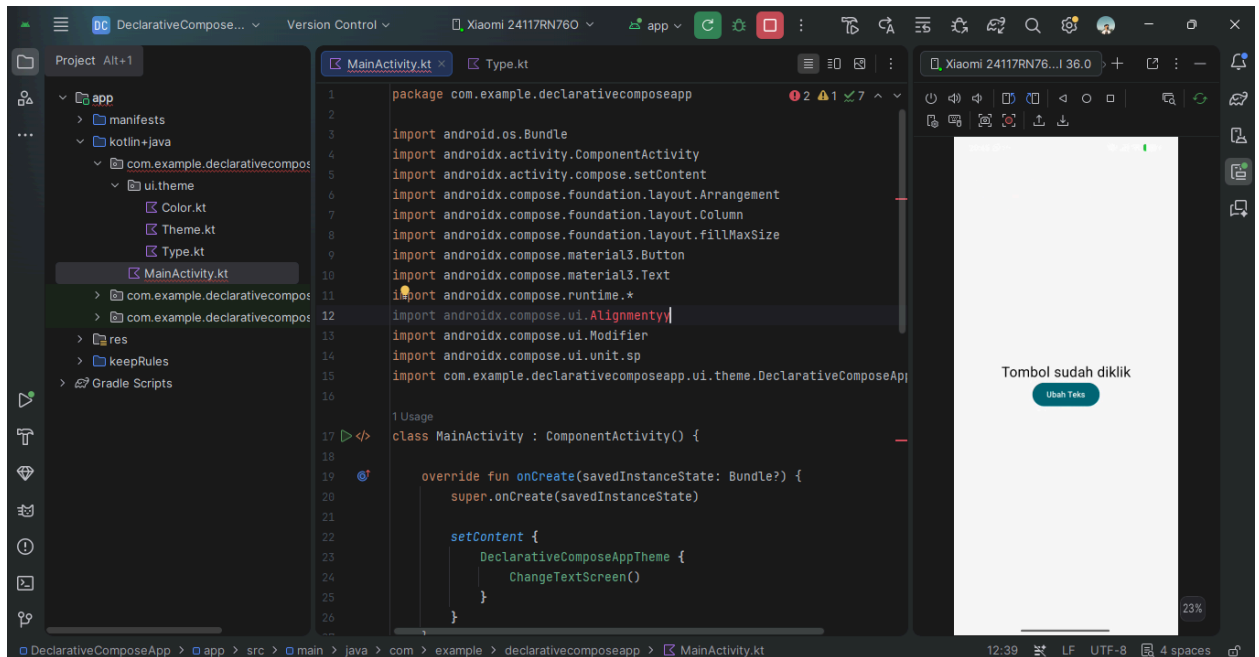


Ketika sudah diubah agar tombol sudah diklik bisa bagian ini :



2. Project B - Jetpack Compose (Deklaratif)





Aktivitas Percobaan :

1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

= Sudah dijawab di soal dan ss diatas

2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

= Objek yang diubah secara langsung adalah TextView dengan ID tvMessage. Perubahan dilakukan dengan mengubah properti text pada objek tersebut.

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

= Nilai yang diubah adalah state message.

4. Mengapa pada Compose tidak terdapat perintah seperti tvMessage.text atau setText()?

= Karena Jetpack Compose menggunakan pendekatan deklaratif. UI ditampilkan berdasarkan nilai state. Ketika state message berubah, Compose melakukan recomposition pada bagian UI yang bergantung pada state tersebut

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

= alurnya adalah tombol di klik, setOnClickListener, tvMessage.text diubah, TextView menampilkan "Tombol sudah diklik"

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

= alurnya adalah tombol di klik message berubah lalu compose melakukan recomposition, lalu text membaca nilai message terbaru lalu nantinya akan muncul UI Menampilkan "Tombol sudah diklik"

7. Apa yang terjadi jika message diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.

= perubahan nilai variabel biasa tidak otomatis diketahui oleh compose sebagai perubahan state. Akibatnya perubahan tersebut tidak memicu recomposition sehingga ui tidak otomatis diperbarui, modul menjelaskan bahwa data yang digunakan untuk memicu recomposition perlu menggunakan state yang dapat diamati seperti `mutableStateOf()`.

8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

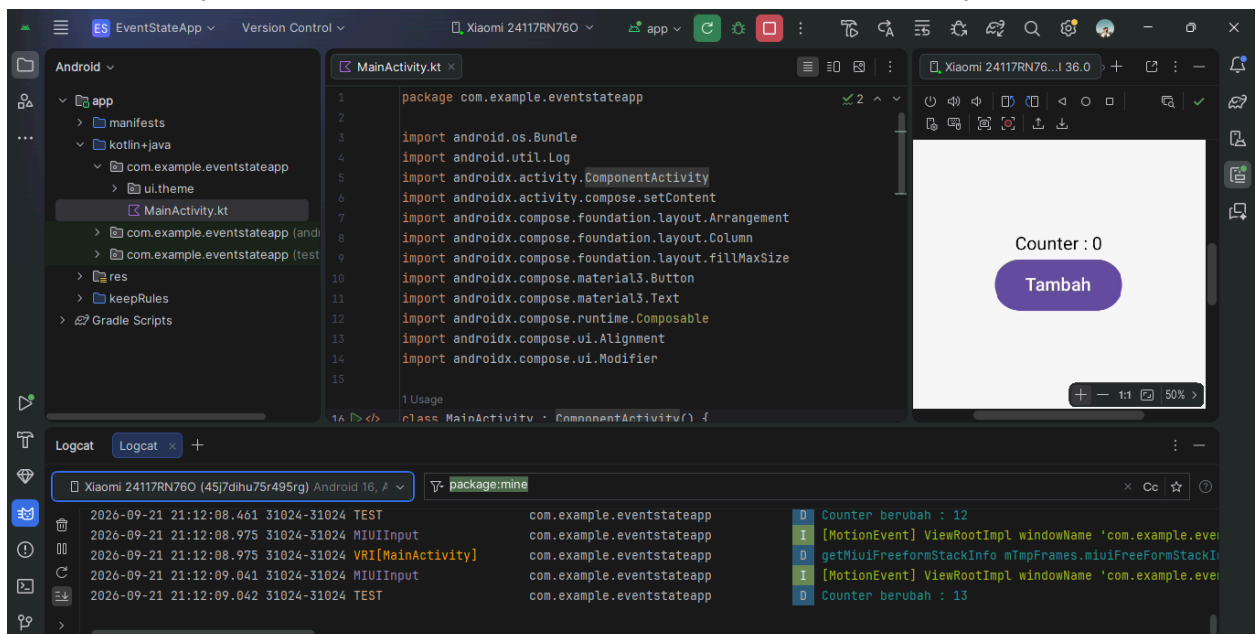
= Pendekatan imperatif memperbaiki komponen UI secara langsung dengan mencari objek View kemudian mengubah propertinya. Sedangkan pendekatan deklaratif pada Jetpack Compose menggunakan state sebagai sumber tampilan. Ketika state berubah, Compose melakukan recomposition sehingga UI menyesuaikan dengan kondisi state terbaru.

Event Handling State

Aktivitas Percobaan :

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

= Logcat menampilkan pesan dengan tag TEST yang menunjukkan perubahan nilai counter, misalnya Counter berubah : 1, Counter berubah : 2, dan seterusnya.



2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?
= Nilai yang tercatat pada Logcat berubah, tetapi tampilan counter tidak otomatis mengikuti perubahan tersebut, nah pada tahap gunakan state tampilan counter ikut berubah sesuai jumlah tombol yang ditekan.
3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?
= Pada tahap pertama, count merupakan variabel biasa sehingga perubahan nilainya tidak menjadi perubahan state yang diamati oleh Compose. Setelah menggunakan remember dan mutableStateOf, Compose dapat mengetahui perubahan state dan memperbarui UI.
4. Apa yang menyebabkan hal tersebut terjadi?
= Perubahan pada variabel biasa tidak secara otomatis diketahui oleh Compose sebagai perubahan state. Oleh karena itu, perubahan nilai count tidak menyebabkan UI melakukan pembaruan atau recomposition. Modul menjelaskan bahwa data yang dapat memicu pembaruan UI pada Compose perlu menggunakan state yang dapat diamati, seperti mutableStateOf().
5. Langkah selanjutnya adalah mengubah kode sebelumnya untuk menggunakan state yaitu pada Composable EventStateTest.
= `var count = 0` diubah jadi `var count by remember { mutableStateOf(0) }`
Sehingga count menjadi state yang dikelola oleh jetpack compose.
6. Ubah kode program pada bagian fungsi Composable EventStateTest dari yang sebelumnya:
= remember digunakan untuk mempertahankan nilai state selama composable masih berada dalam composition, sedangkan mutableStateOf membuat nilai tersebut dapat diamati oleh Compose.

7. Kode tersebut digunakan untuk menjalankan percobaan menggunakan state. Ketika count berubah, Compose dapat mengetahui perubahan tersebut dan memperbarui tampilan.
8. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.
= Program berhasil dijalankan. Setelah tombol "Tambah" diklik beberapa kali, nilai Counter pada tampilan aplikasi ikut bertambah.
Misalnya tombol diklik tiga kali:
Counter : 3
LogCat juga menampilkan perubahan nilai tersebut.
9. LogCat menampilkan pesan dengan tag TEST yang menunjukkan nilai count setelah tombol ditekan.

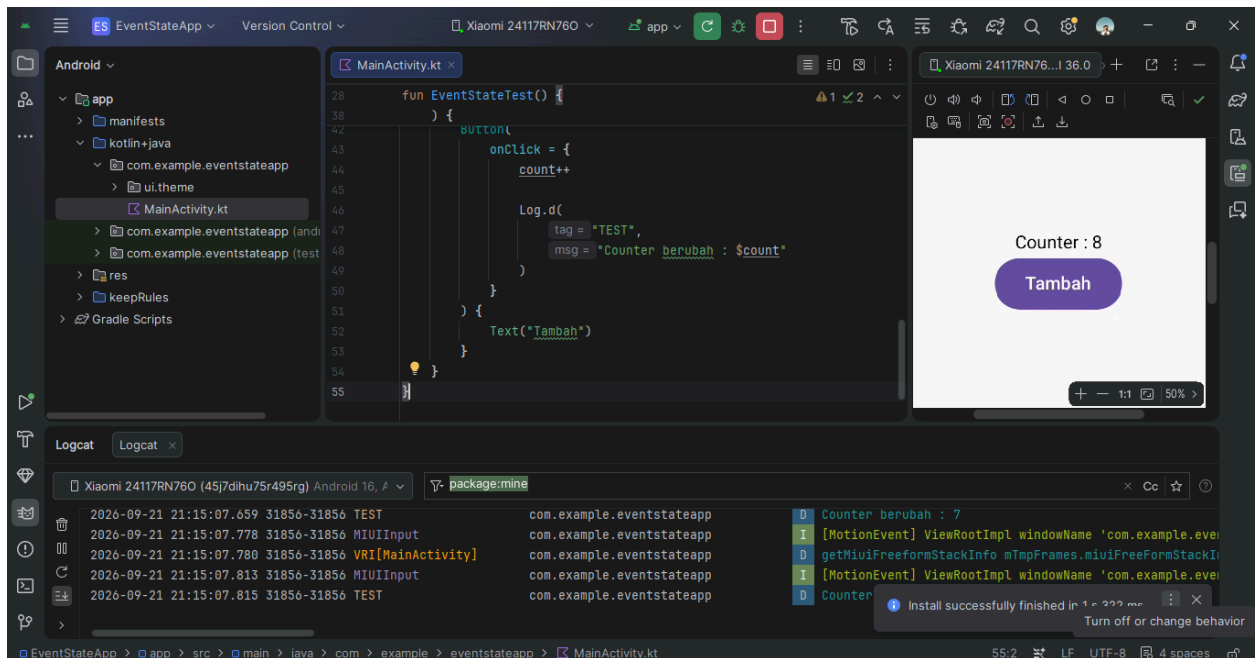
Contohnya:

Counter berubah : 1

Counter berubah : 2

Counter berubah : 3

10. Nilai Counter pada tampilan aplikasi ikut bertambah setiap kali tombol "Tambah" diklik.

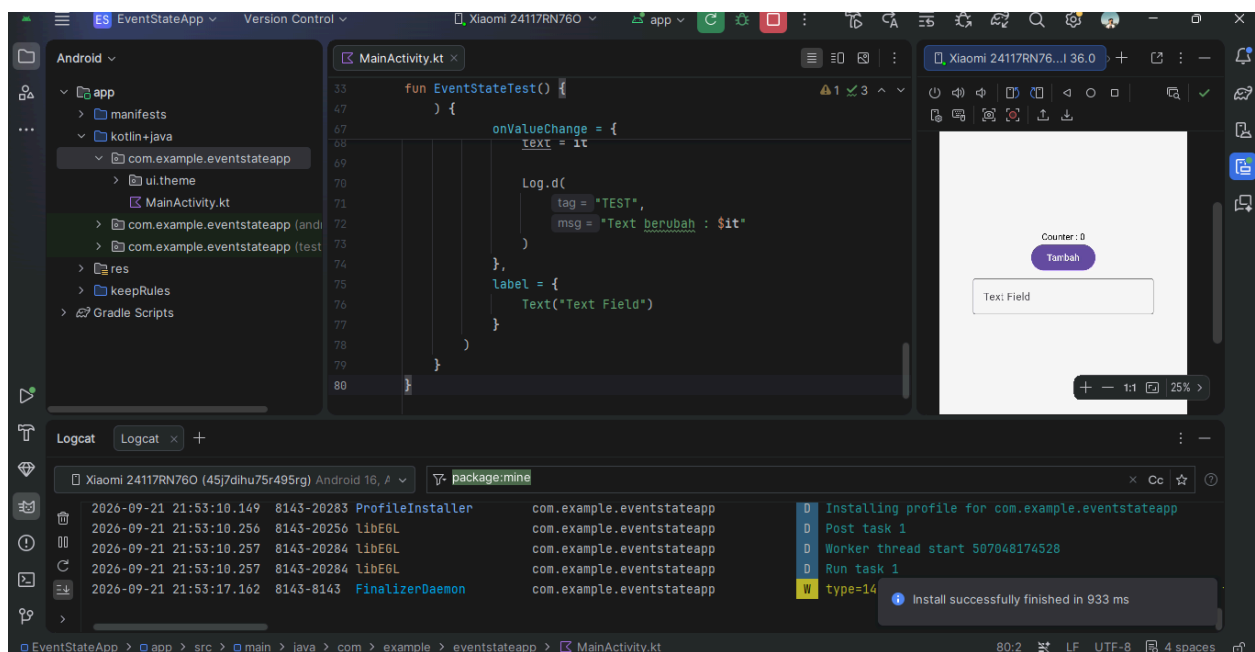


11. = Perbedaan tersebut terjadi karena pada percobaan sebelumnya count merupakan variabel biasa: `var count = 0`, Sedangkan pada percobaan ini count menggunakan state: `var count by remember { mutableStateOf(0) }`. Dengan menggunakan `mutableStateOf`, perubahan nilai count dapat diamati oleh Compose. Ketika nilai state berubah, Compose melakukan recomposition sehingga bagian UI yang menggunakan count diperbarui.

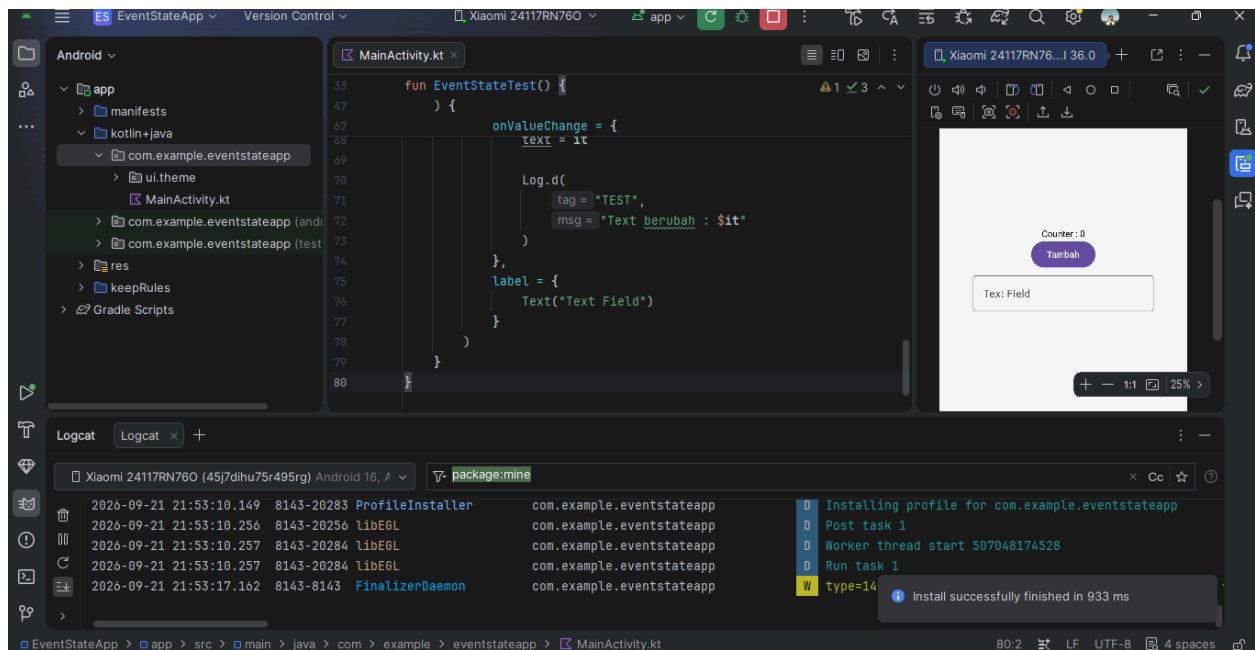
12. Recomposition adalah proses ketika Compose menjalankan kembali bagian dari composable yang bergantung pada state yang mengalami perubahan, sehingga tampilan UI dapat diperbarui sesuai dengan nilai state terbaru.

13. Kode sudah ditambahkan

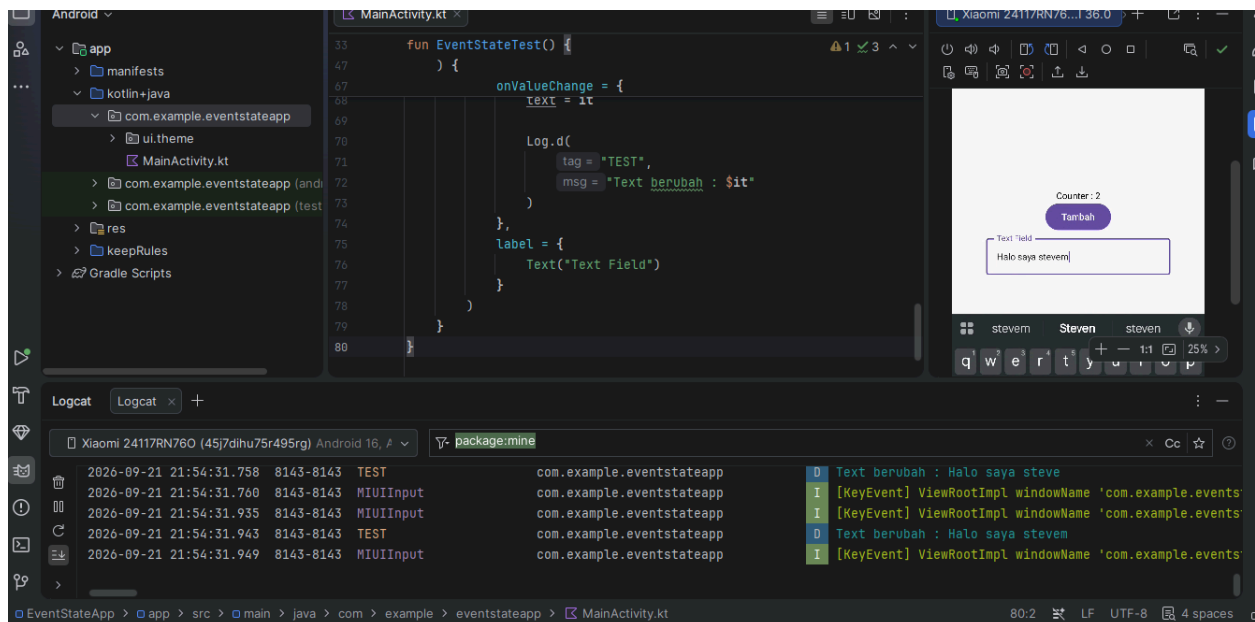
14.



15. Setiap kali pengguna memasukkan atau mengubah huruf pada Text Field, LogCat menampilkan pesan dengan tag TEST dan nilai teks terbaru.



16.



17. Teks yang dimasukkan oleh pengguna akan langsung tampil pada Text Field. Setiap perubahan karakter akan memperbarui nilai yang ditampilkan pada Text Field.

18. Hal tersebut terjadi karena perubahan teks ditangani oleh event onValueChange.

Nilai baru kemudian disimpan ke dalam state text.

19. Terdapat dua state, yaitu:

count, digunakan untuk menyimpan nilai jumlah counter.

`var count by remember { mutableStateOf(0) }`

text, digunakan untuk menyimpan teks yang dimasukkan pada Text Field.

`var text by remember { mutableStateOf("") }`

20.

Terdapat dua event, yaitu:

onClick, pada tombol Tambah.

onValueChange, pada OutlinedTextField.

Keduanya merupakan event handling pada Jetpack Compose. Modul menjelaskan bahwa onClick digunakan untuk menangani aksi ketika tombol ditekan, sedangkan onValueChange digunakan ketika nilai pada input teks berubah.

21. onClick dijalankan ketika pengguna menekan tombol "Tambah". Event tersebut kemudian menambah nilai count dengan:

`count++`

onValueChange dijalankan ketika pengguna memasukkan atau mengubah teks pada Text Field. Event tersebut kemudian menyimpan nilai teks terbaru ke dalam state:

`text = it`

Implisit Intent

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol:
2. Klik Buka Browser
3. Klik Buka Telepon
4. Klik Kirim Email
5. Klik Buka Maps
6. Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya
7. Pada kode yang digunakan, terdapat beberapa action seperti:
8. `Intent.ACTION_VIEW`
9. `Intent.ACTION_DIAL`
10. `Intent.ACTION_SENDTO`
11. Apa perbedaan fungsi dari masing-masing action intent tersebut?
12. Pada kode yang digunakan, terdapat beberapa data seperti:
13. `data = Uri.parse("mailto:test@gmail.com")`
14. `data = Uri.parse("https://www.google.com")`
15. `data = Uri.parse("geo:-7.250445,112.768845")`
16. Apa fungsi data pada intent ?
17. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android

Jawaban :

1. Apa perbedaan fungsi dari masing-masing action intent tersebut?

Intent.ACTION_VIEW digunakan untuk membuka atau menampilkan data, seperti browser dan Maps.

Intent.ACTION_DIAL digunakan untuk membuka aplikasi telepon dengan nomor tertentu.

Intent.ACTION_SENDTO digunakan untuk mengirim data ke tujuan tertentu, seperti alamat email.

2. Apa fungsi data pada intent?

data berfungsi menentukan data atau tujuan yang akan diproses oleh Intent, seperti URL, nomor telepon, alamat email, atau koordinat lokasi.

3. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android?

Action menentukan tindakan yang dilakukan, sedangkan Data menentukan data yang diproses. Android mencocokkan keduanya untuk menentukan aplikasi yang sesuai untuk menjalankan Intent.